

DESIGN AND ANALYSIS OF ALGORITHMS

EXAM-READY STUDY BOOKLET

UNITS I – III COMPREHENSIVE QUESTION SET

Authoritative Source Alignment:

Fundamentals of Computer Algorithms — Ellis Horowitz, Sartaj Sahni, Sanguthevar Rajasekaran (Galgotia Publications)

Preserving exact terminology, recurrences, step counts, algorithms, and solved numericals from classroom PPT lectures.

■ 46 Complete Exam Questions	◆ Solved Numerical Workings
Full answers for every syllabus topic	Prim, Kruskal, Dijkstra, Knapsack, Job Sequencing
◆ Step-by-Step Pseudocode	◆ Asymptotic Recurrences
Exact C/Pascal conventions with step counts	Complete proofs and Master Theorem checks
◆ Educational Vector Diagrams	◆ Last-Minute Revision Sheets
Clean recursion trees, stacks and graphs	Complexity cheat sheet & checklist for exam morning

Targeted for University & College DAA Examinations • 5-Mark & 10-Mark Questions

Generated for Student Academic Excellence • 100% Technical Accuracy

TABLE OF CONTENTS

Comprehensive Structure & Question Mapping

Unit / Section	Topic Description	Questions Covered
UNIT-I: Algorithm Basics & Analysis	Characteristics, Pseudocode conventions, Time & Space complexity $S(P)=C+Sp(I)$, Program step counting (Method I & II Tabular), Best/Worst/Avg Case in Sequential Search, Rate of Growth, Asymptotic Notations (O , Ω , Θ , o), Recursive Factorial & Stack execution, Matrix Addition, Recursive Sum recurrence, Hiring Problem probabilistic analysis, Connected & Biconnected Components, Articulation Points.	Q1 – Q15 (15 Questions)
UNIT-II: Divide and Conquer	General Paradigm, Control Abstraction DAndC(p), Recurrence Relations, Binary Search (Iterative, Recursive, Recurrence & Successful/Unsuccessful analysis), Merge Sort (Algorithm, Divide/Conquer/Combine phases, Call Tree, $O(n \log n)$ Derivation), Quick Sort (Partitioning, Trace, Best $O(n \log n)$ vs Worst $O(n^2)$), Strassen's Matrix Multiplication (7 Products, Formulas, $O(n^{2.81})$ Derivation), General Recurrence & Master Theorem, Detailed Comparison Table.	Q16 – Q30 (15 Questions)
UNIT-III: Greedy Method	Greedy Control Abstraction Greedy(a,n), Greedy-choice property, Feasible vs Optimal solutions, Fractional Knapsack (Formulation, Algorithm, Solved Ratio Numerical), Fractional vs 0/1 Knapsack & Counterexample, Spanning Trees & Properties, Minimum Cost Spanning Tree (MST), Prim's Algorithm (near[] array, Algorithm, $O(n^2)$ complexity), Kruskal's Algorithm (Forest merging, Disjoint Sets Make-Set/Find/Union, $O(E \log E)$), Single-Source Shortest Path (SSSP), Dijkstra's Algorithm (Relaxation, Iteration Table, $O(n^2)$), Job Sequencing with Deadlines (Slot assignment, Algorithms, Solved 4, 5, 7 Job Numericals), Prim vs Kruskal Comparison, Complete Solved Numericals for Prim, Kruskal, Dijkstra, Job Sequencing.	Q31 – Q46 (16 Questions)
FINAL REVISION: Cheat Sheets & Checklist	DAA Last-Minute Revision Sheet, Master Complexity Cheat Sheet (All 15 algorithms), Important Formula Sheet (Asymptotics, Recurrences, Step counts, Knapsack), Important Comparison Tables, Algorithm Selection Guide, and Final Exam Readiness Checklist.	Summary & Checklist

♦ Examination Strategy & Writing Guide

How to use this booklet:

- **For 5-Mark Questions:** Write the Definition, Key Idea, Algorithm block, and Final Complexity.
- **For 10-Mark Questions:** Write the Definition, Complete Pseudocode, Step-by-Step Derivation/Table, Solved Numerical, and Exam Conclusion.
- **On Exam Morning:** Review the *Final Revision Sheet* and *Complexity Cheat Sheet* at the end of the booklet.

UNIT-I: ALGORITHM BASICS, PSEUDOCODE, STEP COUNTING & ANALYSIS OF ALGORITHMS

LAST-MINUTE PRIORITY TOPICS:

- 1. Asymptotic Notations: Big-Oh (O), Omega (Ω), Theta (Θ), Little-oh (o)
- 2. Time Complexity & Program-Step Counting (Method I Count Variable & Method II Tabular Method)
- 3. Space Complexity Components: $S(P) = C + Sp(I)$ (Fixed vs Variable/Stack Space)
- 4. Best, Worst & Average Case Analysis using Sequential Search
- 5. Rate of Growth of Functions & Asymptotic Ordering
- 6. Probabilistic Analysis: The Hiring Problem (Expected Cost $O(ch \ln n)$)
- 7. Graph Basics: Connected Components, Biconnected Components & Articulation Points

[■ VERY HIGH PRIORITY / MUST PREPARE]

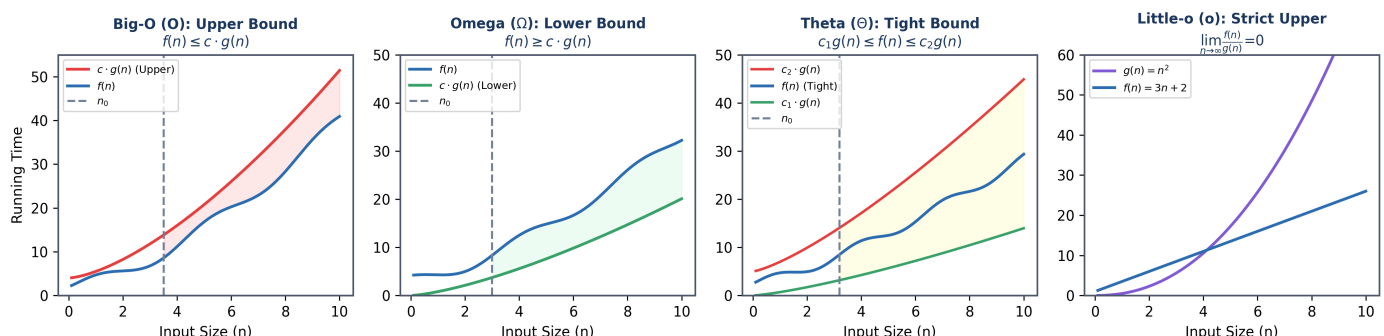
Q1. Explain Asymptotic Notations (O , Ω , Θ , and o) with suitable examples.

1. Need for Asymptotic Analysis

In algorithm analysis, exact execution time varies across machines, compilers, and hardware. **Asymptotic efficiency** analyzes the behavior of algorithm running times as the input size n becomes very large (i.e., for large values of n). It allows us to compare algorithms by focusing on their **rate of growth**, ignoring low-order terms and constant multipliers.

2. Definitions, Formulas & PPT Examples

Notation	Formal Definition	Intuitive Meaning	PPT Examples & Proofs
Big-Oh $O(g(n))$	$f(n) = O(g(n)) \Leftrightarrow \exists$ positive constants c and n_0 such that: $f(n) \leq c \cdot g(n), \forall n \geq n_0$	Upper Bound $f(n)$ grows slower than or at the same rate as $g(n)$. Think of as " $\leq$ ".	$f(n) = 3n + 2 \leq 4n$ ($\forall n \geq 2$) $\Rightarrow O(n)$ $f(n) = 10n^2 + 4n + 2 \leq 11n^2$ ($\forall n \geq 5$) $\Rightarrow O(n^2)$ $f(n) = 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$ ($\forall n \geq 4$) $\Rightarrow O(2^n)$
Omega $\Omega(g(n))$	$f(n) = \Omega(g(n)) \Leftrightarrow \exists$ positive constants c and n_0 such that: $f(n) \geq c \cdot g(n), \forall n \geq n_0$	Lower Bound $f(n)$ grows faster than or at the same rate as $g(n)$. Think of as " $\geq$ ".	$f(n) = 3n + 2 \geq 3n$ ($\forall n \geq 1$) $\Rightarrow \Omega(n)$ $f(n) = 10n^2 + 4n + 2 \geq 1 \cdot n^2$ ($\forall n \geq 1$) $\Rightarrow \Omega(n^2)$ - Also $\Omega(n)$ since $10n^2 + 4n + 2 \geq n$ ($\forall n \geq 0$)
Theta $\Theta(g(n))$	$f(n) = \Theta(g(n)) \Leftrightarrow \exists$ positive constants c_1, c_2, n_0 such that: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq n_0$	Tight Bound $f(n)$ grows at exactly the same rate as $g(n)$. Think of as " $=$ ".	$f(n) = 3n + 2: 3n \leq 3n + 2 \leq 4n$ ($\forall n \geq 2$) $\Rightarrow \Theta(n)$ $f(n) = 10n^2 + 4n + 2: 1n^2 \leq f(n) \leq 11n^2$ ($\forall n \geq 5$) $\Rightarrow \Theta(n^2)$
Little-oh $o(g(n))$	$f(n) = o(g(n)) \Leftrightarrow \lim_{n \rightarrow \infty} [f(n) / g(n)] = 0$ Strict upper bound for all $c > 0$.	Strict Upper Bound $f(n)$ grows strictly slower than $g(n)$. Think of as " $<$ ".	$f(n) = 3n + 2 = o(n^2)$ since $\lim (3n+2)/n^2 = 0$ However, $3n + 2 \neq o(n)$ since limit is $3 \neq 0$



3. Important Asymptotic Properties (Exam Reference)

- Transitivity:** If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.
- Reflexivity:** $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, $f(n) = \Theta(f(n))$.
- Symmetry:** $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$.
- Transpose Symmetry:** $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$.
- Tight Bound Principle:** We always prefer the smallest/tightest upper bound (e.g., $10n^2 + 4n + 2$ is $O(n^2)$, not $O(n^3)$).

◆ Exam Conclusion & Rule of Thumb

Exam Summary: Big-Oh (O) provides worst-case/upper bound guarantee; Omega (Ω) provides lower bound guarantee; Theta (Θ) provides exact/tight growth rate; Little-oh (o) defines strictly non-tight asymptotic dominance.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q2. Explain Time Complexity and Program-Step Counting with suitable examples.

1. Definition of Time Complexity

The **Time Complexity** $T(P)$ of a program P is the sum of its compile time C and its run time $T_P(I)$:

$$T(P) = C + T_P(I)$$

- **Compile time (C):** Does not depend on the instance characteristics (number of inputs n).
- **Run time $T_P(I)$:** Depends directly on the number of inputs (instance characteristics). In DAA, we concentrate on estimating $T_P(I)$.

2. Concept of a Program Step

A **program step** is defined as a syntactically or semantically meaningful program statement whose execution time is independent of the number of inputs.

Examples: $c := a + b$; (1 step), $sum := sum + a[i]$; (1 step).

3. Step-Counting Rules from PPT

1. **Comments:** Contributed step count = 0.
2. **Assignment Statements:** Statements without function calls = 1 step.
3. **Loop Control Statements (for, while, repeat-until):** We consider step counts for the control part. Each execution of the control condition is 1 step, unless expressions depend on instance characteristics.
4. **Conditionals (if-then-else):** Evaluation of condition is 1 step.
5. **Function Calls:** Equal to the number of steps assignable to the function invocation plus 1.

4. Step-Counting Method I: Introduce Global Variable count

■ Method I: Step Count for Iterative Sum

```
Algorithm sum(a, n)
// Iterative sum of n numbers
{
    s := 0;
    count := count + 1;           // for assignment statement s:=0
    for i := 1 to n do
    {
        count := count + 1;       // for 'for' loop condition check
        s := s + a[i];
        count := count + 1;       // for assignment statement
    }
    count := count + 1;           // for last failing check of 'for' (i = n+1)
    return s;
    count := count + 1;           // for return statement
}
// Total Step Count = 1 + n + n + 1 + 1 = 2n + 3 steps -> O(n)
```

◆ Exam Tip

Key Takeaway: Step count reveals exactly how the runtime changes with variations in input size n . For iterative sum, $T(n) = 2n + 3 = O(n)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q3. Explain Space Complexity and its components with formulas and examples.

1. Definition and Formula

The **Space Complexity** of an algorithm is the total amount of memory required by the program to execute to completion.

As per the PPT, the total space $S(P)$ of any program P is divided into two distinct components:

$$S(P) = C + S_P(\text{Instance characteristics})$$

2. Two Space Components

- **i) Fixed Space Requirements (C):** Independent of instance characteristics (input size n). It includes:
 - *Instruction space / Program space:* Memory required to store the compiled machine instructions.
 - *Simple variables and constants:* Fixed space allocated for simple primitive data types and constants.
- **ii) Variable Space Requirements ($S_p(I)$):** Directly depends on instance characteristics I . It includes:
 - *Dynamically allocated memory:* Memory allocated at runtime based on input size n .
 - *Recursive stack space:* Space for formal parameters, local variables, and return addresses pushed to stack during recursive function calls.

3. PPT Solved Examples

Algorithm	Code / Logic	Instance Characteristics Analysis	Variable Space $S_p(I)$
Example 1: Simple Arithmetic $abc(a, b, c)$	<pre> Algorithm abc(a,b,c) { return a+b*b*c+(a+b-c)/(a+b)+4.0; } </pre>	Problem instance is characterized by values a, b, c . Assuming 1 word (4 bytes) stores each variable, memory does not scale with input size.	$S_{abc}(I) = 0$ (Only fixed space C required; $O(1)$)
Example 2: Iterative Sum $sum(a, n)$	<pre> Algorithm sum(a,n) { s:=0; for i:=1 to n do s:=s+a[i]; return s; } </pre>	Problem instance is characterized by n . Space depends on the array a of size n passed to the function.	$S_{sum}(n) = n$ (Linear data space; $O(n)$)
Example 3: Recursive Sum $RSum(a, n)$	<pre> Algorithm RSum(a,n) { if(n≤0) return 0; else return RSum(a,n-1)+a[n]; } </pre>	Per recursive call frame on stack: • formal param a (pointer/ref) = 2 bytes • formal param n (int) = 2 bytes • return address (internal) = 2 bytes Total per call = 6 bytes. Depth = n .	$S_{RSum}(n) = 6n$ (Recursive stack space; $O(n)$)

◆ Space Complexity Conclusion

Exam Tip: In analysis, we focus purely on estimating $S_p(I)$ and ignore the fixed constant C because C remains constant across all input sizes.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q4. Explain Best-Case, Worst-Case and Average-Case Analysis using Sequential Search.

1. Concept & Definitions

All inputs of a given size n do not necessarily take the same number of program steps. Therefore, we categorize performance into three cases:

- **Best-Case Step Count:** Minimum number of steps taken by the algorithm for any input of size n .
- **Worst-Case Step Count:** Maximum number of steps taken by the algorithm for any input of size n .
- **Average-Case Step Count:** Average / expected number of steps taken across all possible inputs of size n .

2. Algorithm: Sequential Search

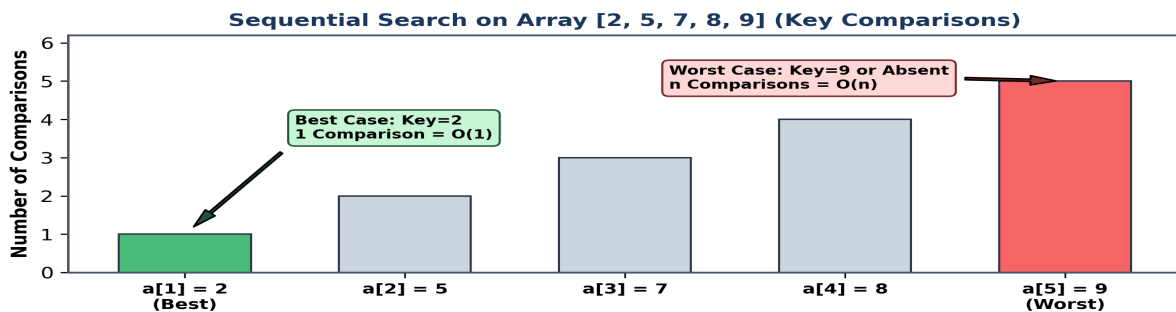
■ Sequential Search Pseudocode

```

Algorithm sequentialSrch(a, n, key)
// Begins at first element and inspects each element in turn until key is found
{
  for i := 1 to n do
  {
    if ( a[i] = key ) then
    {
      return i;    // Key found at index i
    }
  }
  return 0;        // Key not present in array
}
          
```

3. PPT Example Walkthrough on Array $a[] = \{ 2, 5, 7, 8, 9 \}$ ($n = 5$)

Case	Condition / Search Key	Comparisons	Time Complexity
Best Case	Key is at first position (e.g., key = 2). Loop terminates in 1st iteration.	1 comparison	$O(1) / \Theta(1)$
Worst Case	Key is at last position (e.g., key = 9) or key is not present .	n comparisons (5)	$O(n) / \Theta(n)$
Average Case	Key is equally likely to be at any index $1..n$. Sum of steps for all positions divided by total inputs: Avg = $(1 + 2 + 3 + \dots + n) / n = n(n+1)/(2n) = (n+1)/2$	$(n+1)/2$ comparisons (3 for $n=5$)	$O(n) / \Theta(n)$



◆ Conclusion

Exam Conclusion: For sequential search, Best-case = $O(1)$, Worst-case = $O(n)$, and Average-case = $O(n)$. Worst-case analysis is the standard benchmark in computer science because it guarantees an upper bound on running time.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q5. Write an algorithm to find the largest of n numbers and analyze its complexity.

1. Problem Statement & Algorithm

Given an array $a[1:n]$ containing n numbers, find and return the maximum value in the array.

■ Algorithm $\text{max}(a, n)$

```

Algorithm max(a, n)
// a is an array of size n containing numbers
{
    result := a[1];           // Assume first element is maximum (1 step)
    for i := 2 to n do        // Loop from 2nd to nth element (n iterations)
    {
        if ( a[i] > result ) then // Comparison (1 step per iteration)
        {
            result := a[i];       // Update maximum (at most 1 step)
        }
    }
    return result;           // Return largest number (1 step)
}

```

2. Complexity Analysis

- **Number of Comparisons:** The loop runs from $i = 2$ to n , performing exactly **$n - 1$ comparisons**.
- **Best-Case Time:** Maximum is at $a[1]$. Condition $a[i] > \text{result}$ is always false. Time = $(n - 1)$ comparisons = $\Theta(n)$.
- **Worst-Case Time:** Elements are in strictly increasing order. Assignment $\text{result} := a[i]$ executes in every iteration. Time = $\Theta(n)$.
- **Average-Case Time:** $\Theta(n)$.
- **Space Complexity:** Requires **$O(1)$ auxiliary space** (only variables result and i).

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q6. Explain Step-Count Analysis using the Tabular Method with worked examples.

1. Procedure of Tabular Method

In the **Tabular Method**, we create a table with four columns: Statement, steps per execution (**s/e**), **frequency** (number of times executed), and **Total steps** ($s/e \times \text{frequency}$). The total program steps are obtained by summing the total steps column.

2. Worked Example 1: Iterative Sum of n Numbers (PPT Slide 34)

Statement	s/e	Frequency	Total Steps
Algorithm sum(a, n)	0	--	0
{	0	--	0
s := 0;	1	1	1
for i := 1 to n do	1	n + 1	n + 1
s := s + a[i];	1	n	n
return s;	1	1	1
}	0	--	0
Total Steps			2n + 3

3. Worked Example 2: Addition of Two $m \times n$ Matrices (PPT Slide 35)

Statement	s/e	Frequency	Total Steps
Algorithm Add(a, b, c, m, n)	0	--	0
{	0	--	0
for i := 1 to m do	1	m + 1	m + 1
for j := 1 to n do	1	m(n + 1)	mn + m
c[i,j] := a[i,j] + b[i,j];	1	mn	mn
}	0	--	0
Total Steps			2mn + 2m + 1

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q7. Explain Rate of Growth of functions and arrange common functions in increasing order.

1. Rate of Growth Concept

When comparing two algorithms with running times $f(n)$ and $g(n)$, we use rate of growth to characterize how fast each function grows as $n \rightarrow \infty$. **Low-order terms and constants in a function are relatively insignificant for large n.**

2. PPT Dominance Table: $F(n) = n^2 + 100n + \log_{10} n + 1000$ (PPT Slide 42)

n	f(n) Value	n^2 (%)	100n (%)	$\log_{10} n$ (%)	1000 (%)
1	1,101	1 (0.1%)	100 (9.1%)	0 (0.0%)	1000 (90.83%)
10	2,101	100 (4.76%)	1,000 (47.6%)	1 (0.05%)	1,000 (47.60%)
100	21,002	10,000 (47.6%)	10,000 (47.6%)	2 (0.001%)	1,000 (4.76%)
1,000	1,101,003	1,000,000 (90.8%)	100,000 (9.1%)	3 (0.0003%)	1,000 (0.09%)
10,000	101,001,004	100,000,000 (99.0%)	1,000,000 (0.99%)	4 (0.0%)	1,000 (0.001%)

Conclusion from table: As n grows to 10,000, the leading term n^2 contributes **99.0%** of the total value. Hence, $n^2 + 100n + \log_{10} n + 1000 \sim n^2$.

3. Common Functions Ordered by Rate of Growth (Strictly Increasing)

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

Function	Growth Category Name	Value at n = 16	Value at n = 32
1	Constant	1	1
log n	Logarithmic	4	5
n	Linear	16	32

$n \log n$	Log-Linear / $n \log n$	64	160
n^2	Quadratic	256	1,024
n^3	Cubic	4,096	32,768
2^n	Exponential	65,536	4,294,967,296
$n!$	Factorial	2.09×10^{13}	2.63×10^{35}

[■ HIGH PRIORITY]

Q8. Explain the Characteristics of an Algorithm.

Definition: An *algorithm* is a finite set of unambiguous steps used to solve a specific computational problem.

Every valid algorithm must satisfy the following 5 Fundamental Characteristics (PPT Slides 3-4):

1. **Input:** Each algorithm must take **zero or more** externally supplied quantities as input.
2. **Output:** Each algorithm must produce **at least one (or more)** outputs/results.
3. **Definiteness:** Each instruction must be **clear, precise, and unambiguous**.
PPT Counterexamples: Statements such as “add 6 or 7 to x” or “Compute 5/0” are strictly prohibited.
4. **Finiteness:** The algorithm must terminate after a **finite number of steps** for all valid test cases.
5. **Effectiveness:** Every instruction must be basic enough to be carried out **feasibly and exactly in a finite amount of time** by a human using paper and pencil.

[■ HIGH PRIORITY]

Q9. Explain Pseudocode and its conventions / control structures.

Definition: Pseudocode is an informal, high-level description of an algorithm combining structured programming constructs with natural language (looking like C and Pascal) without strict syntax constraints.

10 Pseudocode Conventions from PPT (Slides 5-12):

1. **Comments:** Begin with // and continue until the end of the line.
2. **Blocks:** Indicated with matching braces { and } for compound statements and function bodies.
3. **Data Types:** Not explicitly declared; types and variable scope (global vs. local) are inferred from context.
4. **Assignment Statement:** Represented using the assignment symbol $\langle \text{variable} \rangle := \langle \text{expression} \rangle$.
5. **Boolean & Relational Operators:** Boolean values are true and false. Logical operators: and, or, not. Relational operators: $<, \leq, =, \neq, \geq, >$.
6. **Multidimensional Arrays:** Accessed using square brackets [], e.g., element (i,j) of array A is denoted $A[i,j]$.
7. **Looping Constructs:**
 - while (condition) do { statement_1; ... statement_n; }
 - for variable := value1 to value2 step step do { ... } (step is optional, defaults to +1, can be negative).
 - repeat <statements> until (condition) (executes as long as condition is false; post-test loop).
8. **Conditional Statements:** if <condition> then <statement> and if <condition> then <stmt 1> else <stmt 2>.
9. **Input / Output:** Handled using read and write instructions.
10. **Procedures / Functions:** Starts with the keyword Algorithm Name(<parameter list>) { body }. Simple variables are passed by **value**; arrays and records are passed by **reference**.

[■ HIGH PRIORITY]

Q10. Explain Recursive Algorithms with Factorial as an example.

1. Concept of Recursion

A **recursive algorithm** is a method where a function solves a problem by calling itself on smaller subproblems of the same type until it reaches a terminating **base case**.

2. Algorithm: Recursive Factorial (PPT Slide 16)

■ Algorithm rfactorial(n)

```

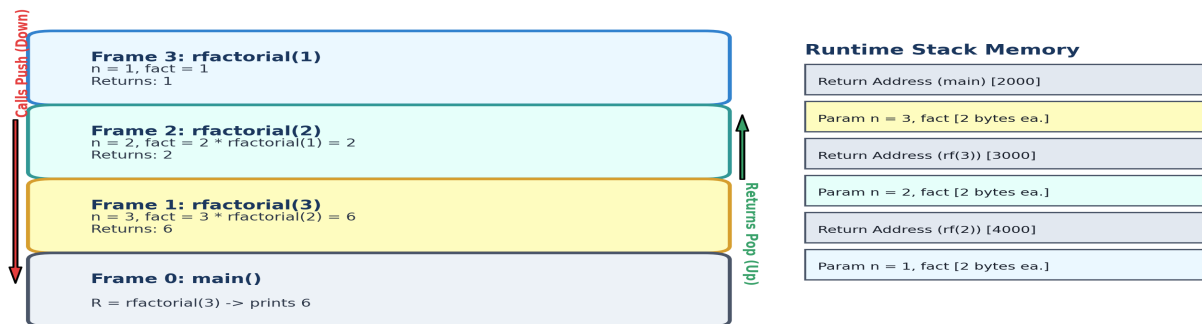
Algorithm rfactorial(n)
// n is an integer >= 0
{
    fact := 1;
    if ( n = 1 or n = 0 ) then
        return fact;           // Base Case
    else
        fact := n * rfactorial(n - 1); // Recursive Call
    return fact;
}

```

3. Stack Execution Mechanism (PPT Slide 17)

Each time a recursive function is called, the current values of parameter n , local variable $fact$, and the **return address** of the next instruction are saved in a stack activation frame. When `main()` invokes `rfactorial(3)`:

- Call 1: pushes $n=3$, return address 2000.
- Call 2: pushes $n=2$, return address 3000.
- Call 3: pushes $n=1$, return address 4000 → hits base case, returns 1.
- Stack pops upward computing: $fact = 2 * 1 = 2$, then $fact = 3 * 2 = 6$, returning 6 to `main()`.



4. Complexity: Time Complexity = $\Theta(n)$; Stack Space Complexity = $\Theta(n)$.

[■ HIGH PRIORITY]

Q11. Find and analyze the step count for the Addition of two matrices.

Problem: Given two $m \times n$ matrices a and b , compute their sum matrix c where $c[i,j] = a[i,j] + b[i,j]$.

■ Algorithm Add(a, b, c, m, n)

```

Algorithm Add(a, b, c, m, n)
{
    for i := 1 to m do           // Executed m + 1 times
    {
        for j := 1 to n do       // Executed m(n + 1) times
        {
            c[i,j] := a[i,j] + b[i,j]; // Executed m * n times
        }
    }
}

```

Step-by-Step Derivation:

- Outer loop for $i := 1$ to m do evaluates condition for $i = 1, 2, \dots, m, m+1 \rightarrow m + 1$ steps.
- Inner loop for $j := 1$ to n do executes $n + 1$ times for each of the m outer iterations $\rightarrow m(n + 1) = mn + m$ steps.
- Inner assignment $c[i,j] := a[i,j] + b[i,j]$ executes exactly mn steps.
- Total Steps = $(m + 1) + (mn + m) + mn = 2mn + 2m + 1$.
- **Asymptotic Complexity:** $\Theta(mn)$. If $m = n$ (square matrices), Total Steps = $2n^2 + 2n + 1 = \Theta(n^2)$.

[■ HIGH PRIORITY]**Q12. Analyze the Recursive Sum algorithm and derive its recurrence relation.****■ Algorithm RSum(a, n)**

```

Algorithm RSum(a, n)
// Recursive sum of first n elements of array a
{
    if ( n <= 0 ) then
        return 0;                // Base case
    else
        return RSum(a, n - 1) + a[n];    // Recursive invocation
}

```

1. Recurrence Formulation (PPT Slide 31):

When analyzing a recursive program for its step count, we obtain a recursive formula:

- If $n = 0$: $T(0) = 2$ (1 for if-condition check + 1 for return 0).
- If $n > 0$: $T(n) = 2 + T(n - 1)$ (1 for if-condition check + 1 for addition/return + $T(n-1)$ for recursive call).

2. Solution by Repeated Substitution (PPT Slide 32):

$$\begin{aligned}
 t_{\text{RSum}}(n) &= 2 + t_{\text{RSum}}(n - 1) \\
 &= 2 + 2 + t_{\text{RSum}}(n - 2) = 2(2) + t_{\text{RSum}}(n - 2) \\
 &= 2(3) + t_{\text{RSum}}(n - 3) \\
 &\dots \\
 &= 2(k) + t_{\text{RSum}}(n - k)
 \end{aligned}$$

Substitute $k = n$:

$$= 2n + t_{\text{RSum}}(0) = 2n + 2$$

The exact step count for RSum is $2n + 2 \Rightarrow O(n)$.

[■ HIGH PRIORITY]**Q13. Explain Probabilistic Analysis using the Hiring Problem.****1. Problem Scenario (PPT Slide 59)**

You use an employment agency to hire a new office assistant. The agency sends one candidate each day for n days. You interview each candidate and decide immediately whether to hire them. You want to have the best candidate seen so far at all times. If a candidate is better than the current assistant, you fire the current assistant and hire the new candidate. You always hire candidate 1.

■ Hire-Assistant(n) Pseudocode

```

Hire-Assistant(n)
{
    best := 0;                // Candidate 0 is a dummy least-qualified candidate
    for i := 1 to n do
    {
        interview candidate i;    // Cost c_i per candidate
        if candidate i is better than candidate best then
        {
            best := i;
            hire candidate i;      // Cost c_h per candidate
        }
    }
}

```

2. Cost Model & Cases (PPT Slides 60-65)

- Cost to interview: c_i ; Cost to hire: c_h . Total cost = $n \cdot c_i + m \cdot c_h$ (where m = number of hires).
- **Best Case:** Best candidate arrives on day 1 $\rightarrow m = 1 \rightarrow \text{Cost} = n \cdot c_i + c_h$.
- **Worst Case:** Candidates arrive in strictly increasing order of quality $\rightarrow$ hire all $n \rightarrow \text{Cost} = n \cdot c_i + n \cdot c_h$.

3. Average-Case Probabilistic Analysis (PPT Slides 63-65)

- There are $n!$ equally likely input permutations.
- Probability $P(i)$ that candidate i is better than all previous $i - 1$ candidates is $1 / i$.

PPT Example ($n = 4, i = 3$): Out of 24 permutations of {1,2,3,4}, exactly 8 have candidate 3 greater than candidate 1 and 2 $\rightarrow 8/24 = 1/3$.

- Expected number of candidates hired: $E[m] = \sum_{i=1}^n P(i) = \sum_{i=1}^n (1/i) = \ln n + 1$.
- Expected Hiring Cost = $O(c_h \ln n)$. Total expected cost = $O(n \cdot c_i + c_h \ln n)$.

[■ HIGH PRIORITY]

Q14. Explain Connected Components in an undirected graph.

1. Definition: A **Connected Component** in an undirected graph G is a maximal subgraph in which every pair of vertices is connected by a path (mutually reachable).

2. Key Characteristics (PPT Slide 67):

- A connected component is an equivalence class of vertices under the reachability relation.
- An undirected graph can have multiple disconnected components.
- Inside a component, each vertex is reachable from every other vertex in that component, and no edges exist connecting vertices across different components.

3. Identification Algorithm (PPT Slide 68):

Connected components are identified using **Depth-First Search (DFS)** or **Breadth-First Search (BFS)** in $O(V + E)$ time.

4. PPT Example (Slide 66): In an undirected graph with vertices {1, 2, 3, 4, 5, 6, 7}:

- Component 1 = {1, 2, 3, 4}
- Component 2 = {5, 6}
- Component 3 = {7} (isolated vertex)

[■ HIGH PRIORITY]

Q15. Explain Biconnected Components and Articulation Points with examples.

1. Definitions (PPT Slide 69):

- **Articulation Point (Cut Vertex):** A vertex in a connected graph whose removal (along with all edges incident to it) disconnects the remaining graph into two or more connected components.
- **Biconnected Component (BCC):** A maximal subgraph that remains connected even after removing any single vertex (i.e., contains no articulation points).

2. PPT Solved Example (Slides 70-71):

Consider a connected graph with vertices {0, 1, 2, 3, 4, 5, 6, 7, 8} where vertex 4 acts as a bridge junction between two cycles {0,1,2,3} and {5,6,7,8}:

- **Articulation Point: Vertex 4** (removing vertex 4 splits the graph into two disconnected components: {0,1,2,3} and {5,6,7,8}).
- Removing any other vertex (e.g., 0, 1, 2, or 3) does not disconnect the graph because alternative paths exist.
- **Resulting Biconnected Components:**
 1. BCC 1 = {0, 1, 2, 3, 4}
 2. BCC 2 = {4, 5, 6, 7, 8}

3. Algorithmic Determination: DFS using discovery time $dfn[u]$ and lowest reachable ancestor $low[u]$ in $O(V + E)$ time. Root is an articulation point if it has ≥ 2 children; non-root u is an articulation point if $low[v] \geq dfn[u]$ for some child v .

UNIT-II: DIVIDE AND CONQUER TECHNIQUE & RECURRENCE RELATIONS

LAST-MINUTE PRIORITY TOPICS:

- 1. Divide and Conquer Paradigm & Control Abstraction DAndC(p)
- 2. Binary Search: Iterative, Recursive, Recurrence & Successful/Unsuccessful Analysis
- 3. Merge Sort: Divide, Conquer, Combine, Full Merge Algorithm & $O(n \log n)$ Derivation
- 4. Quick Sort: Partitioning Procedure, Trace & Best $O(n \log n)$ vs Worst $O(n^2)$ Derivation
- 5. Strassen's Matrix Multiplication: 7 Products $M \blacksquare M \blacksquare$ & $O(n^{2.81})$ Recurrence Derivation
- 6. General Recurrence Relations $T(n) = aT(n/b) + f(n)$ & Master Theorem
- 7. Comprehensive Comparison: Merge Sort vs Quick Sort

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q16. Explain the Divide and Conquer Technique and its Control Abstraction.

1. General Method & Concept (PPT Slides 2-3)

The **Divide and Conquer** technique is a top-down algorithmic paradigm that solves a computational problem by breaking it into smaller instances of the same problem. It operates in three distinct phases:

- **Divide:** Splits n inputs into k subsets (where $1 < k \leq n$), resulting in k smaller subproblems.
- **Conquer:** If subproblems are small enough (base case), solve them directly. If subproblems are large, recursively re-apply Divide and Conquer.
- **Combine:** Combines the solutions of the k subproblems using a separate procedure to obtain the solution to the original whole problem.

2. Control Abstraction: Algorithm DAndC(p) (PPT Slide 4)

■ Control Abstraction for Divide and Conquer

```

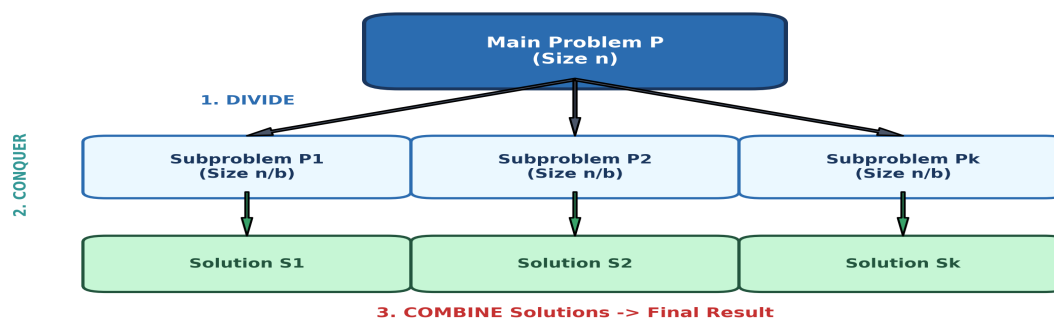
Algorithm DAndC(p)
// p is the given problem instance
{
    if Small(p) then
        return s(p);                                // Base case: solve directly
    else
    {
        divide p into smaller problems p1, p2, ..., pk, k >= 1; // Divide phase
        Apply DAndC to each of these subproblems;                // Conquer phase (recursive)
        return Combine(DAndC(p1), DAndC(p2), ..., DAndC(pk)); // Combine phase
    }
}

```

3. General Computing Time Recurrence (PPT Slide 5)

If the size of problem p is n and the sizes of the k subproblems are $n_1, n_2, \dots, n_k$, the computing time $T(n)$ is described by:

- $T(n) = g(n)$, when n is small (time to compute answer directly)
- $T(n) = T(n_1) + T(n_2) + \dots + T(n_k) + f(n)$, otherwise (where $f(n)$ is time to divide and combine)



◆ Control Abstraction Rule

Exam Key: The recurrence for homogeneous subproblems takes the form $T(n) = c$ (for n small) and $T(n) = aT(n/b) + f(n)$ (for $n > 1$), where $a \geq 1$, $b > 1$, and n is a power of b ($n = b^k$).

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q17. Explain Binary Search using Divide and Conquer and analyze its complexity.

1. Problem Statement & Principle

Given a sorted array $a[1:n]$ in non-decreasing order ($a[1] \leq a[2] \leq \dots \leq a[n]$) and a search element x , determine whether x is present in the array. If present, return its index mid ; otherwise, return 0.

■ Algorithm BinSrch(a , low , $high$, x)

Algorithm BinSrch(a , low , $high$, x)

// Recursive Divide and Conquer Binary Search on $a[low : high]$

```
{
    if ( low = high ) then                // If Small(P): size 1 base case
    {
        if ( x = a[low] ) then return low;
        else return 0;
    }
    else
    {
        mid := (low + high) / 2;           // Divide: compute middle index
        if ( x = a[mid] ) then return mid; // Key found
        else if ( x < a[mid] ) then
            return BinSrch(a, low, mid - 1, x); // Conquer: search left subproblem
        else
            return BinSrch(a, mid + 1, high, x); // Conquer: search right subproblem
    }
}
```

2. Time Complexity Recurrence & Derivation (PPT Slide 10)

If the time for dividing the list is constant c_2 and base case is c_1 :

• $T(n) = c_1$ (for $n = 1$)

• $T(n) = T(n/2) + c_2$ (for $n > 1$)

Assume $n = 2^k$ (where $k = \log_2 n$):

$T(n) = T(n/2) + c_2 = T(n/4) + c_2 + c_2 = T(n/8) + 3c_2 = \dots = T(n/2^k) + k \cdot c_2$

$T(n) = T(1) + k \cdot c_2 = c_1 + c_2 \log_2 n = O(\log n)$.

3. Complexity Summary Table (PPT Slide 11)

Search Outcome	Best Case	Average Case	Worst Case
Successful Searches	$O(1)$ (found at first mid)	$O(\log n)$	$O(\log n)$
Unsuccessful Searches	$O(\log n)$	$O(\log n)$	$O(\log n)$

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q18. Explain Merge Sort with complete algorithm and a step-by-step example.

1. Concept (PPT Slide 12):

- **Base Case:** Solve directly if array has 1 element (already sorted).
- **Divide:** Divide array into two equal halves (size $n/2$).
- **Conquer:** Recursively sort the two halves.
- **Combine:** Merge the two sorted subarrays into a single sorted array.

■ Algorithm MergeSort(low , $high$)

```

Algorithm MergeSort(low, high)
// Sorts global array a[low : high] in ascending order
{
    if ( low < high ) then                // If more than one element
    {
        mid := (low + high) / 2;          // Divide
        MergeSort(low, mid);              // Conquer left half
        MergeSort(mid + 1, high);         // Conquer right half
        Merge(low, mid, high);            // Combine: merge two sorted subsets
    }
}

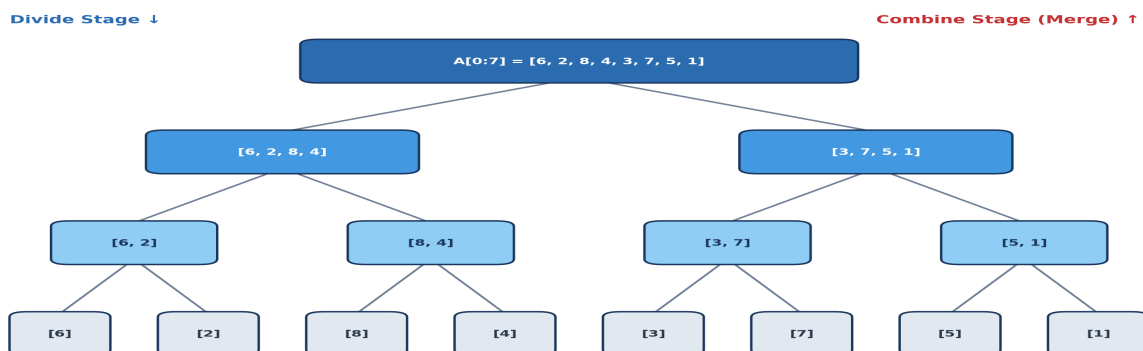
```

2. Step-by-Step PPT Example: Array A = [6, 2, 8, 4, 3, 7, 5, 1] (Slides 14-35)

• **Divide Stage:** [6, 2, 8, 4 | 3, 7, 5, 1] → [6, 2 | 8, 4] & [3, 7 | 5, 1] → leaves [6], [2], [8], [4], [3], [7], [5], [1].

• **Merge Stage (Bottom-Up):**

1. Merge(6, 2) → [2, 6] and Merge(8, 4) → [4, 8]
2. Merge([2, 6], [4, 8]) → [2, 4, 6, 8] (Left half fully sorted)
3. Merge(3, 7) → [3, 7] and Merge(5, 1) → [1, 5]
4. Merge([3, 7], [1, 5]) → [1, 3, 5, 7] (Right half fully sorted)
5. **Final Merge:** Merge([2, 4, 6, 8], [1, 3, 5, 7]) → [1, 2, 3, 4, 5, 6, 7, 8].



3. Tree of Calls Example (PPT Slide 36): For array of 10 elements [179, 254, 285, 310, 351, 423, 450, 520, 652, 861], root call is (1, 10), split into (1, 5) and (6, 10) down to base calls (1,1), (2,2), ... (10,10).

Complexity: Best = $O(n \log n)$, Average = $O(n \log n)$, Worst = $O(n \log n)$. Auxiliary Space = $O(n)$ for temporary array b[].

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q19. Analyze Merge Sort using Recurrence Relation and derive $O(n \log n)$.

1. Formulation of Recurrence (PPT Slide 52)

If the time for the merging operation is proportional to n (i.e., $c_2 n$), the computing time for Merge Sort is described by:

- $T(n) = c_1$, for $n = 1$ (c_1 is a constant)
- $T(n) = 2T(n/2) + c_2 n$, for $n > 1$ (c_2 is a constant)

2. Step-by-Step Derivation by Repeated Substitution

Assume $n = 2^k$, which implies $k = \log_2 n$:

$$\begin{aligned}
 T(n) &= 2T(n/2) + c_2 n \\
 &= 2 [2T(n/4) + c_2(n/2)] + c_2 n = 4T(n/4) + 2c_2 n \\
 &= 4 [2T(n/8) + c_2(n/4)] + 2c_2 n = 8T(n/8) + 3c_2 n
 \end{aligned}$$

...

$$= 2^k T(n / 2^k) + k \cdot c_2 n$$

Substitute $n = 2^k$ (so $n / 2^k = 1$ and $k = \log_2 n$):

$$\begin{aligned}
 T(n) &= 2^k T(1) + (\log_2 n) \cdot c_2 n \\
 &= c_1 n + c_2 n \log_2 n \\
 &= O(n \log n).
 \end{aligned}$$

◆ Master Theorem Cross-Check

Exam Verification (Master Theorem): For $T(n) = 2T(n/2) + \Theta(n)$, $a = 2$, $b = 2$, $d = 1$. Since $\log_b a = \log_2 2 = 1 = d \Rightarrow$ Case 2 applies $\Rightarrow T(n) = \Theta(n^d \log n) = \Theta(n \log n)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q20. Explain Quick Sort with Partitioning Procedure and analyze its complexity.

1. Principle & Divide-Conquer Steps (PPT Slide 54)

- **Divide:** Pick any element as the **pivot** (e.g., first element $a[\text{low}]$). Partition remaining elements into:
 - *FirstPart*: contains all elements $\leq$ pivot
 - *SecondPart*: contains all elements $>$ pivot
- **Conquer:** Recursively sort FirstPart and SecondPart using QuickSort.
- **Combine:** *No work is necessary* because partitioning and sorting are performed in-place.

2. PPT Partition Trace: Array [85, 24, 63, 95, 17, 31, 45, 98] (Slides 57-60)

- **Initial:** Pivot = **85**, pointer i starts at left (moves right while $a[i] <$ pivot), pointer j starts at right (moves left while $a[j] >$ pivot).
- i stops at 95 (> 85), j stops at 45 (< 85). Since $i < j$, **swap 95 and 45** $\rightarrow$ [85, 24, 63, 45, 17, 31, 95, 98].
- Advance pointers: i advances to index 7 (95), j decreases to index 6 (31). Now $i \geq j$ (pointers crossed).
- **Swap pivot (85) with $a[j]$ (31):** Array becomes [31, 24, 63, 45, 17 | 85 | 95, 98].
- **Result:** Pivot 85 is in its correct final position at index 6, yielding two independent sublists: FirstPart = [31, 24, 63, 45, 17] and SecondPart = [95, 98].

■ Algorithm QuickSort(low, high)

```
Algorithm QuickSort(low, high)
// Sorts global array a[low:high] into ascending order; a[high+1] is sentinel  $\geq$  all elements
{
    if ( low < high ) then
    {
        j := Partition(low, high);           // j is position of pivot element
        QuickSort(low, j - 1);               // Recursively sort left subarray
        QuickSort(j + 1, high);              // Recursively sort right subarray
    }
}
```

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q21. Explain Best-Case and Worst-Case Complexity of Quick Sort with derivations.

1. Worst-Case Analysis: $O(n^2)$ (PPT Slides 65-67)

- **Condition:** Occurs when the input list is **already sorted** in ascending or descending order.
- **Behavior:** In every step, the pivot is the smallest (or largest) element. Partitioning creates one empty sublist and one sublist of size $n - 1$.
- **Recurrence:**

$$\begin{aligned}
 T(n) &= T(n-1) + c_2 n \\
 &= T(n-2) + c_2(n-1) + c_2 n \\
 &= T(n-3) + c_2(n-2) + c_2(n-1) + c_2 n = \dots \\
 &= c_2 [1 + 2 + 3 + \dots + n] = c_2 \cdot [n(n+1)/2] = O(n^2).
 \end{aligned}$$

2. Best-Case Analysis: $O(n \log n)$ (PPT Slides 68-70)

- **Condition:** Occurs when each partition divides the list into **two equal-sized sublists** (size $\approx n/2$).
- **Recurrence:**

$$T(n) = 2T(n/2) + c_2 n \text{ (identical to Merge Sort)}$$
 Solving by repeated substitution: $T(n) = 2^k T(1) + k \cdot c_2 n = c_1 n + c_2 n \log_2 n = O(n \log n)$.
- **Space Complexity:** **Best/Average stack space** = $O(\log n)$; **Worst-case stack space** = $O(n)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q22. Explain Strassen's Matrix Multiplication algorithm with formulas and motivation.

1. Motivation & Standard Method (PPT Slides 73-77)

Standard matrix multiplication for two $n \times n$ matrices uses 3 nested loops: $C[i,j] = \sum A[i,k] \cdot B[k,j]$, requiring n^3 multiplications $\Rightarrow O(n^3)$.

A direct Divide and Conquer splits matrices into four $n/2 \times n/2$ blocks, requiring **8 multiplications** and 4 additions $\Rightarrow T(n) = 8T(n/2) + c_2 n^2 \Rightarrow O(n^3)$ (no improvement).

Strassen's Breakthrough: Matrix multiplications are computationally much more expensive than additions ($O(n^3)$ vs $O(n^2)$). Strassen discovered a way to compute $C = AB$ using only **7 matrix multiplications** and **18 matrix additions/subtractions**.

2. The 7 Strassen Multiplication Formulas (M_1 to M_7) (PPT Slide 78)

- $M_1 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$
- $M_2 = (A_{21} + A_{22}) \times B_{11}$
- $M_3 = A_{11} \times (B_{12} - B_{22})$
- $M_4 = A_{22} \times (B_{21} - B_{11})$
- $M_5 = (A_{11} + A_{12}) \times B_{22}$
- $M_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$
- $M_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$

3. Formulas for Resulting Matrix C Quadrants (PPT Slide 78)

- $C_{11} = M_1 + M_4 - M_5 + M_7$
- $C_{12} = M_3 + M_5$
- $C_{21} = M_2 + M_4$
- $C_{22} = M_1 + M_3 - M_2 + M_6$

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q23. Analyze Strassen's Matrix Multiplication using recurrence relation and derive $O(n^{2.81})$.

1. Formulation of Recurrence (PPT Slide 79)

Since Strassen performs 7 multiplications on matrices of size $n/2$ and 18 additions of size $n/2 \times n/2$ (costing $c_2 n^2$):

- $T(n) = c_1$, for $n \leq 2$
- $T(n) = 7T(n/2) + c_2 n^2$, for $n > 2$

2. Derivation by Repeated Substitution (PPT Slide 80)

Assume $n = 2^k$, so $k = \log_2 n$:

$$T(n) = 7^k T(1) + c_2 n^2 [1 + (7/4) + (7/4)^2 + (7/4)^3 + \dots + (7/4)^{k-1}]$$

Using the geometric series formula $S_k = a(r^n - 1) / (r - 1)$ with $r = 7/4 > 1$:

$$\begin{aligned} &= 7^{\log_2 n} c_1 + c_2 n^2 \cdot (7/4)^{\log_2 n} \\ &= c_1 n^{\log_2 7} + c_2 n^{\log_2 4} \cdot (n^{\log_2 7 - \log_2 4}) \\ &= c_1 n^{\log_2 7} + c_2 n^{\log_2 7} \\ &= O(n^{\log_2 7}) \approx O(n^{2.807}) \approx O(n^{2.81}). \end{aligned}$$

◆ Performance Note

Exam Comparison: Standard Matrix Multiplication is $O(n^3)$. Strassen's Algorithm is $O(n^{2.81})$. For large matrices ($n > 1000$), Strassen provides massive speedups.

[■ HIGH PRIORITY]

Q24. Explain the General Recurrence Relation used in Divide and Conquer and Master Theorem.

1. Standard Form: $T(n) = a T(n/b) + f(n)$

- $a \geq 1$: Number of subproblems generated.
- $b > 1$: Factor by which the input size is reduced.

- **f(n)**: Time required to divide the problem and combine subproblem solutions.

2. Master Theorem Rules (for $f(n) = \Theta(n^d)$):

- **Case 1** ($a > b^d$): $T(n) = \Theta(n^{\log_b a})$ (dominated by leaf subproblems).
- **Case 2** ($a = b^d$): $T(n) = \Theta(n^d \log n)$ (equal work at all tree levels).
- **Case 3** ($a < b^d$): $T(n) = \Theta(n^d)$ (dominated by root divide/combine work).

[■ HIGH PRIORITY]

Q25. Write the Iterative Binary Search Algorithm.

■ Iterative Binary Search

```
Algorithm BinSearch(a, n, x)
// a is an array of size n in ascending order, x is the key to search
{
    low := 1; high := n;
    while ( low <= high ) do
    {
        mid := (low + high) / 2;
        if ( x < a[mid] ) then high := mid - 1;
        else if ( x > a[mid] ) then low := mid + 1;
        else return mid;           // Key found at mid
    }
    return 0;                     // Key not found
}
```

[■ HIGH PRIORITY]

Q26. Write the Recursive Binary Search Algorithm.

■ Recursive Binary Search

```
Algorithm BinSrch(a, low, high, x)
// Given array a[low:high] in increasing order, return j such that x=a[j], else 0
{
    if ( low = high ) then           // If Small(P)
    {
        if ( x = a[low] ) then return low;
        else return 0;
    }
    else
    {
        mid := (low + high) / 2;
        if ( x = a[mid] ) then return mid;
        else if ( x < a[mid] ) then
            return BinSrch(a, low, mid - 1, x);
        else
            return BinSrch(a, mid + 1, high, x);
    }
}
```

[■ HIGH PRIORITY]

Q27. Explain the Divide, Conquer and Combine Steps of Merge Sort.

1. **Divide**: The array $a[\text{low}:\text{high}]$ of size n is divided into two halves by finding the middle position $\text{mid} = (\text{low} + \text{high})/2$ in **$O(1)$ time**.
2. **Conquer**: The two subarrays $a[\text{low}:\text{mid}]$ and $a[\text{mid}+1:\text{high}]$ are sorted independently by recursive invocations of MergeSort until subproblems contain 1 element.
3. **Combine**: The two sorted subarrays are merged into a single sorted array by Merge(low , mid , high) in **$O(n)$ time** using an auxiliary buffer array $b[]$.

[■ HIGH PRIORITY]

Q28. Write the complete Merge() algorithm.

■ Algorithm Merge(low, mid, high)

```

Algorithm Merge(low, mid, high)
// a[low:mid] and a[mid+1:high] are two sorted subsets in global array a.
// b[] is a temporary global array. Merges two sets into a[low:high].
{
    h := low; j := mid + 1; i := low;
    while ( h <= mid ) and ( j <= high ) do
    {
        if ( a[h] <= a[j] ) then { b[i] := a[h]; h := h + 1; }
        else { b[i] := a[j]; j := j + 1; }
        i := i + 1;
    }
    if ( h > mid ) then
        for k := j to high do { b[i] := a[k]; i := i + 1; }
    else
        for k := h to mid do { b[i] := a[k]; i := i + 1; }
    for k := low to high do a[k] := b[k]; // Copy back to original array
}

```

[■ HIGH PRIORITY]

Q29. Write the complete Quick Sort and Partition algorithms.

■ Algorithm Partition & Interchange

```

Algorithm Partition(low, high)
{
    pivot := a[low]; i := low; j := high + 1;
    while ( i < j ) do
    {
        i++;
        while ( a[i] < pivot ) do i++;
        j--;
        while ( a[j] > pivot ) do j--;
        if ( i < j ) then Interchange(i, j); // Swap out-of-place elements
    }
    Interchange(low, j); // Place pivot in correct position
    return j;
}

Algorithm interchange(x, y)
{
    temp := a[x]; a[x] := a[y]; a[y] := temp;
}

```

[■ HIGH PRIORITY]

Q30. Compare Merge Sort and Quick Sort in detail.

Feature / Metric	Merge Sort	Quick Sort
Paradigm	Divide and Conquer	Divide and Conquer
Best-Case Time	$O(n \log n)$	$O(n \log n)$
Average-Case Time	$O(n \log n)$	$O(n \log n)$
Worst-Case Time	$O(n \log n)$ (Guaranteed)	$O(n^2)$ (Already sorted)
Auxiliary Space	$O(n)$ (Requires buffer array b)	$O(\log n)$ stack space (In-place)
Sorting Location	Not in-place (requires extra array)	In-place sorting
Stability	Stable (preserves duplicate order)	Unstable (swaps across pivot)
Primary Work Phase	In the Combine (Merge) phase	In the Divide (Partition) phase
Data Structure Preference	Excellent for Linked Lists & External files	Faster in practice for contiguous Arrays

UNIT-III: GREEDY METHOD, SPANNING TREES, SHORTEST PATHS & JOB SEQUENCING

LAST-MINUTE PRIORITY TOPICS:

- 1. Greedy Method & Control Abstraction Greedy(a, n)
- 2. Fractional Knapsack: Formulation, Algorithm & Solved Profit-Ratio Numerical
- 3. Fractional vs 0/1 Knapsack Comparison & Counterexample
- 4. Minimum Cost Spanning Trees: Prim's Algorithm (near[] array) & Kruskal's Algorithm
- 5. Disjoint Sets: Make-Set, Find-Set & Union Operations
- 6. Single-Source Shortest Path (SSSP): Dijkstra's Algorithm & Numerical Iteration Table
- 7. Job Sequencing with Deadlines: Greedy Slot Selection & Solved 4, 5 and 7-Job Numericals
- 8. Comprehensive Numerical Workings for Prim, Kruskal, Dijkstra & Job Sequencing

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q31. Explain the Greedy Method and its Control Abstraction.

1. Principle & Core Characteristics (PPT Slides 2-3)

The **Greedy Method** is a problem-solving strategy that constructs an optimal solution through a sequence of decisions made one by one in some order.

- **Greedy-Choice Property:** At each step, the algorithm makes the choice that looks best at the moment (locally optimal choice) without reconsidering previous decisions.
- **Irrevocable Decisions:** A decision, once made, is usually never changed later.
- **Feasible Solution:** A subset of inputs that satisfies all given problem constraints.
- **Optimal Solution:** A feasible solution that optimizes (maximizes or minimizes) the given objective function.

2. Greedy Control Abstraction / General Method (PPT Slide 4)

■ Greedy Control Abstraction

```

Algorithm Greedy(a, n)
// a[1:n] contains the n inputs
{
    solution := empty;                // Initialize empty solution set
    for i := 1 to n do
    {
        x := Select(a);              // Select next item using greedy criterion
        if Feasible(solution, x) then // Check if adding x satisfies constraints
            solution := Union(solution, x); // Include x into solution set
    }
    return solution;
}
  
```

3. PPT Example & Limitation (PPT Slides 5-10):

- *Largest k-out-of-n Sum:* Greedily picking the largest available number at each step is always optimal.
- *Shortest Paths on a Multi-Stage Graph:* A greedy local decision at stage 1 may lead to extremely expensive edges in subsequent stages; hence, greedy does not work for all graph topologies.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q32. Explain Fractional Knapsack using the Greedy Method with algorithm and numerical.

1. Problem Formulation (PPT Slide 11)

Given n items with profits $p_i > 0$ and weights $w_i > 0$, and a knapsack of capacity m . We can take fractions of items ($0 \leq x_i \leq 1$).

Maximize: $\sum_{i=1}^n p_i x_i$ **Subject to:** $\sum_{i=1}^n w_i x_i \leq m$.

Greedy Decision Property: Sort and select items in **decreasing order of profit/weight ratio** (p_i / w_i).

■ Algorithm GreedyKnapsack(m, n)

```

Algorithm GreedyKnapsack(m, n)
// p[1:n] and w[1:n] contain profits and weights ordered such that p[i]/w[i] >= p[i+1]/w[i+1]
// m is knapsack capacity and x[1:n] is the output solution vector
{
    for i := 1 to n do x[i] := 0;           // Initialize solution vector
    U := m;                                // Remaining knapsack capacity
    for i := 1 to n do
    {
        if ( w[i] > U ) then break;        // Cannot take full item
        x[i] := 1;
        U := U - w[i];                     // Deduct weight
    }
    if ( i <= n ) then x[i] := U / w[i];    // Take fractional portion of item i
}
// Time Complexity = O(n) if already sorted, O(n log n) with sorting.

```

2. PPT Solved Numerical Example (PPT Slides 12-13)

Given: $n = 5$ items, knapsack capacity $m = 10$ ml.

Weights: $w = [4, 8, 2, 6, 1]$ | Profits: $p = [\$12, \$32, \$40, \$30, \$50]$.

Item (i)	Weight (w_i)	Profit (p_i)	Ratio (p_i/w_i)	Greedy Rank	Amount Taken (x_i)	Profit Earned
Item 5	1 ml	\$50	50	1st	1 ml ($x_5 = 1$)	\$50
Item 3	2 ml	\$40	20	2nd	2 ml ($x_3 = 1$)	\$40
Item 4	6 ml	\$30	5	3rd	6 ml ($x_4 = 1$)	\$30
Item 2	8 ml	\$32	4	4th	1 ml ($x_2 = 1/8$)	$\$32 \times 1/8 = \4
Item 1	4 ml	\$12	3	5th	0 ml ($x_1 = 0$)	\$0
Total	10 ml	--	--	--	Solution Vector: (0, 1/8, 1, 1, 1)	\$124

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q33. Compare Fractional Knapsack and 0/1 Knapsack with counterexample.

1. Fundamental Difference: In **Fractional Knapsack**, items are divisible ($0 \leq x_i \leq 1$), and the greedy method always yields an optimal solution. In **0/1 Knapsack**, items are indivisible ($x_i \in \{0, 1\}$); greedy strategy fails because choosing by ratio may leave unusable empty space.

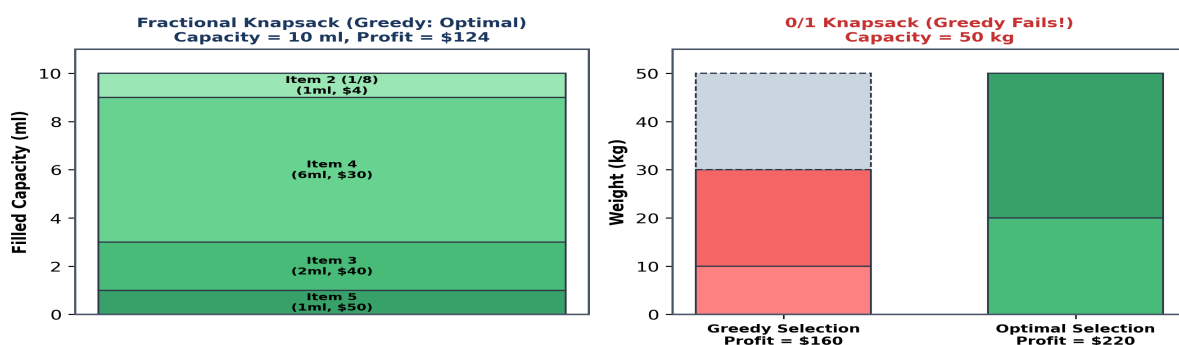
2. PPT Counterexample Proving Greedy Fails for 0/1 Knapsack (PPT Slides 17-18)

Given: Capacity $m = 50$ kg. 3 Items:

- Item 1: Weight = 10 kg, Profit = \$60 → Ratio = 6
- Item 2: Weight = 20 kg, Profit = \$100 → Ratio = 5
- Item 3: Weight = 30 kg, Profit = \$120 → Ratio = 4

• **Greedy Approach:** Picks Item 1 (10 kg, \$60) and Item 2 (20 kg, \$100). Total weight = 30 kg. Remaining 20 kg cannot accommodate Item 3 (30 kg). **Total Profit = \$160.**

• **Optimal 0/1 Solution:** Takes Item 2 (20 kg, \$100) and Item 3 (30 kg, \$120). Total weight = 50 kg. **Total Profit = \$220.**



◆ Key Distinction

Exam Rule: Fractional Knapsack is solved in $O(n \log n)$ using Greedy Method. 0/1 Knapsack cannot be solved optimally by Greedy; it requires Dynamic Programming ($O(n \cdot m)$) or Branch & Bound.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q34. Define Spanning Tree and explain its properties.

Definition (PPT Slide 19): A *tree* is a connected undirected graph that contains no cycles. A **Spanning Tree** of a connected undirected graph $G = (V, E)$ is a subgraph that is a tree and contains **all the vertices of G**.

4 Key Properties (PPT Slide 20):

1. For an n -vertex connected graph, any spanning tree contains exactly **$n - 1$ edges**.
2. It connects all n vertices in the graph.
3. A spanning tree is minimally connected and contains **no cycles**.
4. Adding any non-tree edge forms exactly one cycle; removing any tree edge disconnects the graph.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q35. Explain Minimum Cost Spanning Tree (MST) and its applications.

Definition (PPT Slide 22): For a weighted connected undirected graph, a **Minimum Cost Spanning Tree (MST)** is a spanning tree whose sum of edge weights is the smallest among all possible spanning trees:

$\text{Cost}(T) = \sum_{(u,v) \in T} \text{cost}(u, v)$ is minimized.

Applications (PPT Slide 23):

- **Computer Networks & Wiring:** Finding how to connect a set of computers/routers using the minimum amount of wire.
- **Infrastructure:** Pipeline laying, electrical power grid routing, telephone line wiring, and transportation networks.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q36. Explain Prim's Algorithm with example, algorithm and complexity.**1. Principle & Procedure (PPT Slides 24-26)**

Prim's algorithm starts with the **minimum cost edge** (k, l) in the graph and sets $\text{near}[k] = \text{near}[l] = 0$. At each of the next $n - 2$ steps, it selects an edge $(j, \text{near}[j])$ such that vertex j is not yet in the tree and $\text{cost}[j, \text{near}[j]]$ is minimum among all unvisited vertices. It adds $(j, \text{near}[j])$ to the MST, sets $\text{near}[j] = 0$, and updates $\text{near}[k]$ for remaining vertices.

■ Algorithm Prim(E, cost, n, t)

```

Algorithm Prim( $E, \text{cost}, n, t$ )
//  $E$  is edge set,  $\text{cost}[1:n, 1:n]$  is cost matrix,  $t[1:n-1, 1:2]$  stores MST edges
{
    Let  $(k, l)$  be an edge of minimum cost in  $E$ ;
    mincost := cost[ $k, l$ ];
     $t[1, 1] := k$ ;  $t[1, 2] := l$ ;
     $\text{near}[k] := \text{near}[l] := 0$ ;
    for  $i := 1$  to  $n$  do // Initialize near[]
        if (  $\text{near}[i] \neq 0$  ) then
            if (  $\text{cost}[i, k] < \text{cost}[i, l]$  ) then  $\text{near}[i] := k$ ;
            else  $\text{near}[i] := l$ ;
    for  $i := 2$  to  $n - 1$  do // Find  $n - 2$  additional edges
    {
        Let  $j$  be index such that  $\text{near}[j] \neq 0$  and  $\text{cost}[j, \text{near}[j]]$  is minimum;
         $t[i, 1] := j$ ;  $t[i, 2] := \text{near}[j]$ ;
        mincost := mincost + cost[ $j, \text{near}[j]$ ];
         $\text{near}[j] := 0$ ; // Add vertex  $j$  to tree
        for  $k := 1$  to  $n$  do // Update near[]
            if (  $\text{near}[k] \neq 0$  ) and (  $\text{cost}[k, \text{near}[k]] > \text{cost}[k, j]$  ) then
                 $\text{near}[k] := j$ ;
    }
    return mincost;
}

```

2. Time Complexity Analysis (PPT Slide 30)

- Line 8 (find minimum edge): $O(E)$.
- Line 12 initialization loop: $O(n)$.
- The outer loop (Line 14) iterates $n - 2$ times. In each iteration, finding minimum $\text{near}[j]$ (Lines 17-18) takes $O(n)$ time, and updating $\text{near}[j]$ (Line 22) takes $O(n)$ time.
- Total time = $(n - 2) \times O(n) = O(n^2)$. Using adjacency matrix, Prim's complexity is $O(n^2)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]**Q37. Explain Kruskal's Algorithm with example, algorithm and complexity.****1. Principle & Procedure (PPT Slides 31-33)**

Kruskal's algorithm starts with a forest of n disjoint single-vertex trees. It sorts all edges in non-decreasing order of cost. It then iterates through the sorted edges, adding the next edge (u, v) to the forest **if and only if it does not create a cycle** (verified using $\text{Find-Set}(u) \neq \text{Find-Set}(v)$). Once selected, it performs $\text{Union}(j, k)$ to merge the components.

■ Algorithm kruskal(E, cost, n, t)

```

Algorithm kruskal(E, cost, n, t)
// Constructs MST stored in t[1:n-1, 1:2]
{
    for i := 1 to n do Make-Set(i);           // Each vertex in a separate set
    Sort the edges of E into increasing order by cost;
    mincost := 0; i := 1;
    for each edge (u, v) in E in increasing order of cost do
    {
        j := Find-Set(u); k := Find-Set(v);
        if ( j != k ) then                    // Does not form a cycle
        {
            t[i,1] := u; t[i,2] := v;
            mincost := mincost + cost[u, v];
            Union(j, k);                      // Merge the two components
            i := i + 1;
        }
    }
    return mincost;
}

```

2. Complexity (PPT Slide 39): Edge sorting takes $O(E \log E)$. With Disjoint Sets (Union-Find), the total running time is dominated by sorting $\Rightarrow O(E \log E) = O(E \log V)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]**Q38. Explain Disjoint Sets, FIND-SET and UNION operations.**

1. Definition (PPT Slide 34): Two sets A and B are disjoint if they share no common elements, i.e., $A \cap B = \emptyset$.

PPT Example: $S_1 = \{1, 7, 8, 9\}$, $S_2 = \{2, 5, 10\}$, and $S_3 = \{3, 4, 6\}$ are three mutually disjoint sets.

2. Core Operations (PPT Slide 35):

- **Make-Set(x):** Creates a new set containing only element x, with representative x.
 - **FIND-SET(x):** Returns the representative element of the set containing x.
 - **UNION(i, j):** Combines the two disjoint sets represented by i and j into a single set and selects a new representative.
- Role in Kruskal:** An edge (u, v) creates a cycle if and only if $\text{FIND-SET}(u) == \text{FIND-SET}(v)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]**Q39. Explain the Single-Source Shortest Path (SSSP) problem.**

Problem Definition (PPT Slide 40): Given a positively weighted directed graph $G = (V, E)$ with non-negative edge costs ($\text{cost}(e) \geq 0$) and a distinguished source vertex v, find the shortest directed path from v to every other vertex in the graph.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q40. Explain Dijkstra's Algorithm with pseudocode, worked example and complexity.

1. Concept (PPT Slide 42): Dijkstra's algorithm maintains a set S of vertices whose shortest path from source v is finalized, and an array $\text{dist}[]$ of current shortest estimates. In each step, it picks $u \notin S$ with minimum $\text{dist}[u]$, adds u to S , and **relaxes** all outgoing edges from u :

if ($\text{dist}[w] > \text{dist}[u] + \text{cost}[u,w]$) then $\text{dist}[w] := \text{dist}[u] + \text{cost}[u,w]$;

■ Algorithm ShortestPaths (Dijkstra)

```

Algorithm ShortestPaths(v, cost, dist, n)
// dist[j] stores shortest distance from v to j. cost is adjacency matrix.
{
    for i := 1 to n do { s[i] := false; dist[i] := cost[v, i]; }
    s[v] := true; dist[v] := 0;           // Source vertex placed in S
    for num := 2 to n do
    {
        Choose u from vertices not in S such that dist[u] is minimum;
        s[u] := true;                     // Put u into S
        for each w adjacent to u with s[w] = false do
            if ( dist[w] > dist[u] + cost[u, w] ) then
                dist[w] := dist[u] + cost[u, w]; // Relaxation
    }
}

```

2. PPT Solved 6-Vertex Digraph Trace Table (PPT Slides 40-41)

Iteration	S Set	Selected u	dist[2]	dist[3]	dist[4]	dist[5]	dist[6]
Initial	{1}	--	50	10 ✓	∞	45	∞
Iter 1	{1, 3}	v_3 (10)	50	10	25 ✓	45	∞
Iter 2	{1, 3, 4}	v_4 (25)	45	10	25	45	28 ✓
Iter 3	{1, 3, 4, 6}	v_6 (28)	45	10	25	45 ✓	28
Iter 4	{1, 3, 4, 5, 6}	v_5 (45)	45 ✓	10	25	45	28

Final Shortest Paths: $v_1 \rightarrow v_3$ (10), $v_1 \rightarrow v_3 \rightarrow v_4$ (25), $v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_6$ (28), $v_1 \rightarrow v_3 \rightarrow v_4 \rightarrow v_2$ (45), $v_1 \rightarrow v_5$ (45). **Time Complexity:** $O(n^2)$.

[■ VERY HIGH PRIORITY / MUST PREPARE]

Q41. Explain Job Sequencing with Deadlines using Greedy Method.**1. Problem Definition (PPT Slide 46)**

Given n jobs, each with deadline $d_i \geq 1$ and profit $p_i > 0$. Each job takes 1 unit of time on a single processor. Profit is earned iff the job finishes by its deadline. **Objective:** Find a feasible processing schedule that maximizes total profit.

2. Greedy Strategy & Algorithm (PPT Slides 49-52)

- Sort all jobs in **descending order of profit** ($p_1 \geq p_2 \geq \dots \geq p_n$).
- For each job i , assign it to the **latest available free time slot** $r \leq d_i$. If all slots $\leq d_i$ are occupied, discard the job.

■ Algorithm JS(d, j, n)

```

Algorithm JS(d, j, n)
// Jobs ordered by p[1] >= p[2] >= ... >= p[n]. j[i] is ith job in optimal schedule
{
    d[0] := j[0] := 0; j[1] := 1; k := 1;
    for i := 2 to n do
    {
        r := k;
        while ( (d[j[r]] > d[i]) and (d[j[r]] > r) ) do r := r - 1;
        if ( d[i] > r ) then
        {
            for q := k to (r + 1) step -1 do j[q + 1] := j[q];
            j[r + 1] := i;
            k := k + 1;
        }
    }
    return k;
}
// Worst-case Time Complexity = O(n^2); with Disjoint Sets = O(n alpha(n)).

```

3. PPT Worked Examples Summary (PPT Slides 48, 50):

- **Ex 1 (n=4):** p = (100, 10, 15, 27), d = (2, 1, 2, 1) → Optimal = {J₁, J₄} → Max Profit = **127**.
- **Ex 2 (n=5):** p = (20, 15, 10, 5, 1), d = (2, 2, 1, 3, 3) → Optimal = {J₁, J₂, J₄} → Max Profit = **40**.
- **Ex 3 (n=7):** p = (3, 5, 20, 18, 1, 6, 30), d = (1, 3, 4, 3, 2, 1, 2) → Optimal = {J₆, J₇, J₄, J₃} → Max Profit = **74**.

[■ HIGH PRIORITY]

Q42. Compare Prim's and Kruskal's Algorithms in detail.

Feature / Metric	Prim's Algorithm	Kruskal's Algorithm
Basic Approach	Tree Growing: Starts at a single root and grows one connected tree.	Forest Merging: Starts with disjoint vertices and adds globally cheapest edges.
Starting Point	Starts with minimum cost edge (or arbitrary start vertex).	Starts with globally minimum edge anywhere in the graph.
Graph Type Preference	Preferred for Dense Graphs (where $E \approx V^2$).	Preferred for Sparse Graphs (where $E \ll V^2$).
Time Complexity	$O(V^2)$ with matrix; $O(E \log V)$ with Min-Heap.	$O(E \log E) = O(E \log V)$ (dominated by sorting).
Cycle Check Method	Maintains $near[] = 0$ for tree vertices (implicit).	Explicit cycle checking via Disjoint Sets (Find-Set) .
Data Structures	Arrays $near[]$, $cost[][]$ or Priority Queue.	Edge array, Disjoint Sets (Make-Set, Find, Union).

[■ HIGH PRIORITY]

Q43. Solve an MST Numerical Problem using Prim's Algorithm.

Problem: Find the Minimum Cost Spanning Tree for a 6-vertex graph with edges: (1,2):4, (1,3):2, (2,3):1, (2,4):5, (3,4):8, (3,5):10, (4,5):2, (4,6):6, (5,6):3.

Step	Selected Edge	Tree Vertices (S)	Candidate Edges Considered	Cost Added	Total Cost
Init	(2, 3)	{2, 3}	Cheapest edge in graph (cost 1)	1	1
1	(3, 1)	{1, 2, 3}	(1,3):2, (1,2):4, (2,4):5, (3,4):8, (3,5):10	2	3
2	(2, 4)	{1, 2, 3, 4}	(2,4):5, (3,4):8, (3,5):10	5	8
3	(4, 5)	{1, 2, 3, 4, 5}	(4,5):2, (4,6):6, (3,5):10	2	10
4	(5, 6)	{1, 2, 3, 4, 5, 6}	(5,6):3, (4,6):6	3	13

Final MST Edges: {(2,3), (1,3), (2,4), (4,5), (5,6)}. **Minimum Total Cost = 13.**

[■ HIGH PRIORITY]

Q44. Solve an MST Numerical Problem using Kruskal's Algorithm.

Problem: Apply Kruskal's Algorithm to the same 6-vertex graph. First, sort all edges by weight:

(2,3):1, (1,3):2, (4,5):2, (5,6):3, (1,2):4, (2,4):5, (4,6):6, (3,4):8, (3,5):10.

Edge Examined	Cost	Find-Set(u)	Find-Set(v)	Action / Cycle Status	MST Total Cost
(2, 3)	1	{2}	{3}	ACCEPTED (Union {2, 3})	1
(1, 3)	2	{1}	{2, 3}	ACCEPTED (Union {1, 2, 3})	3
(4, 5)	2	{4}	{5}	ACCEPTED (Union {4, 5})	5
(5, 6)	3	{4, 5}	{6}	ACCEPTED (Union {4, 5, 6})	8
(1, 2)	4	{1, 2, 3}	{1, 2, 3}	REJECTED (Forms Cycle 1-3-2)	8
(2, 4)	5	{1, 2, 3}	{4, 5, 6}	ACCEPTED (Union all → 5 edges done)	13

Verification: Both Prim and Kruskal yield identical minimum cost = 13.

[■ HIGH PRIORITY]

Q45. Solve a Dijkstra Shortest-Path Numerical Problem with step-by-step table.

Problem: Given a 5-vertex directed graph with source $v_1 = 1$.

Edges: (1,2):10, (1,4):5, (2,3):1, (2,4):2, (4,2):3, (4,3):9, (4,5):2, (5,1):7, (5,3):6, (3,5):4.

Step	Selected u	S Set	dist[1]	dist[2]	dist[3]	dist[4]	dist[5]
Init	--	{1}	0	10	∞	5 ✓	∞
1	v_4 (5)	{1, 4}	0	$\min(10, 5+3)=8$	$\min(\infty, 5+9)=14$	5	$\min(\infty, 5+2)=7$ ✓
2	v_5 (7)	{1, 4, 5}	0	8 ✓	$\min(14, 7+6)=13$	5	7
3	v_2 (8)	{1, 4, 5, 2}	0	8	$\min(13, 8+1)=9$ ✓	5	7
4	v_3 (9)	{1, 4, 5, 2, 3}	0	8	9	5	7

Shortest Distances from 1: dist[2]=8 (via 1→4→2), dist[3]=9 (via 1→4→2→3), dist[4]=5 (direct), dist[5]=7 (via 1→4→5).

[■ HIGH PRIORITY]

Q46. Solve a Job Sequencing with Deadlines Numerical Problem with slot assignment.

Problem: Given 5 jobs with profits and deadlines: J_1 :(p=20, d=2), J_2 :(p=15, d=2), J_3 :(p=10, d=1), J_4 :(p=5, d=3), J_5 :(p=1, d=3).

Max deadline $d_{\max} = 3$ slots (Slot 1: [0,1], Slot 2: [1,2], Slot 3: [2,3]).

Job (Sorted)	Profit	Deadline	Slot Assigned	Gantt Chart State	Cumulative Profit
J_1	20	2	Slot 2 [1, 2]	[− , J_1 , −]	20
J_2	15	2	Slot 1 [0, 1]	[J_2 , J_1 , −]	35
J_3	10	1	REJECTED (Slot 1 full)	[J_2 , J_1 , −]	35
J_4	5	3	Slot 3 [2, 3]	[J_2 , J_1 , J_4]	40
J_5	1	3	REJECTED (All full)	[J_2 , J_1 , J_4]	40

Optimal Job Schedule: [J_2 , J_1 , J_4]. Maximum Earned Profit = 15 + 20 + 5 = 40.

FINAL REVISION: DAA LAST-MINUTE REVISION SHEET, FORMULAS & EXAM CHECKLIST

LAST-MINUTE PRIORITY TOPICS:

- 1. Comprehensive Complexity Cheat Sheet (All Units I-III Algorithms)
- 2. Master Formula Sheet: Asymptotics, Recurrences, Step Counts & Knapsack
- 3. Key Differences: O vs Ω vs Θ vs o , Merge vs Quick, Prim vs Kruskal, Frac vs 0/1
- 4. Quick Algorithm Selection Guide: Choosing the Right Algorithmic Tool
- 5. Itemized Final University Exam Checklist

1. DAA Master Complexity Cheat Sheet (Units I – III)

A quick-reference table containing exact complexities supported by the course material:

Algorithm / Problem	Paradigm	Best Case	Average Case	Worst Case	Aux. Space
Sequential Search	Brute Force	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Find Largest (Max)	Iterative	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Iterative Sum	Iterative	$2n + 3$ steps	$2n + 3$ steps	$O(n)$	$O(n)$ data
Recursive Sum	Recursive	$2n + 2$ steps	$2n + 2$ steps	$O(n)$	$O(n)$ stack
Matrix Addition	Nested Loops	$2mn+2m+1$	$O(mn)$	$O(mn)$	$O(mn)$ data
Binary Search (Iter.)	Divide & Conquer	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Binary Search (Rec.)	Divide & Conquer	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$ stack
Merge Sort	Divide & Conquer	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ array
Quick Sort	Divide & Conquer	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ stack
Strassen's Matrix Mult.	Divide & Conquer	$O(n^{2.81})$	$O(n^{2.81})$	$O(n^{\log_2 7})$	$O(n^2)$
Fractional Knapsack	Greedy Method	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Prim's Algorithm (MST)	Greedy Method	$O(V^2)$	$O(V^2)$	$O(V^2)$	$O(V)$
Kruskal's Algorithm	Greedy Method	$O(E \log E)$	$O(E \log E)$	$O(E \log E)$	$O(V + E)$
Dijkstra's SSSP	Greedy Method	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$
Job Sequencing	Greedy Method	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(n)$

2. Master Exam Formula Sheet

◆ Essential Formulas & Recurrences

1. Asymptotic Limit & Inequality Definitions:

- **Big-Oh:** $f(n) \leq c \cdot g(n)$, $\forall n \geq n_0 \Rightarrow$ Upper Bound.
- **Omega:** $f(n) \geq c \cdot g(n)$, $\forall n \geq n_0 \Rightarrow$ Lower Bound.
- **Theta:** $c_1 g(n) \leq f(n) \leq c_2 g(n)$, $\forall n \geq n_0 \Rightarrow$ Tight Bound.
- **Little-oh:** $\lim_{n \rightarrow \infty} [f(n) / g(n)] = 0 \Rightarrow$ Strict Non-Tight Upper Bound.
- **Growth Ordering:** $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$

2. Space & Time Complexity Formulas:

- **Time Complexity:** $T(P) = C + T_p(\text{Instance characteristics})$
- **Space Complexity:** $S(P) = C + S_p(\text{Instance characteristics})$
- **Iterative Sum Steps:** $2n + 3$ | **Recursive Sum Steps:** $2n + 2$ | **Matrix Add Steps:** $2mn + 2m + 1$
- **Hiring Problem Cost:** Total Cost = $n \cdot c_i + m \cdot c_h$ | Expected Hires $E[m] = \ln n + 1$ | Expected Cost = $O(c_h \ln n)$

3. Recurrence Relations:

- **Binary Search:** $T(n) = T(n/2) + c_2 \Rightarrow T(n) = O(\log n)$
- **Merge Sort:** $T(n) = 2T(n/2) + c_2 n \Rightarrow T(n) = O(n \log n)$
- **Quick Sort (Worst):** $T(n) = T(n-1) + c_2 n \Rightarrow T(n) = O(n^2)$
- **Quick Sort (Best):** $T(n) = 2T(n/2) + c_2 n \Rightarrow T(n) = O(n \log n)$
- **Strassen Matrix Mult:** $T(n) = 7T(n/2) + c_2 n^2 \Rightarrow T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$
- **Master Theorem** ($T(n) = a T(n/b) + \Theta(n^d)$): Case 1 ($a > b^d \Rightarrow \Theta(n^{\log_b a})$); Case 2 ($a = b^d \Rightarrow \Theta(n^d \log n)$); Case 3 ($a < b^d \Rightarrow \Theta(n^d)$).

4. Greedy Problem Objectives:

- **Fractional Knapsack:** Maximize $\sum p_i x_i$ subject to $\sum w_i x_i \leq m$ ($0 \leq x_i \leq 1$). Greedy criterion: sort by p_i / w_i .
- **MST Property:** Exactly $n - 1$ edges for an n -vertex graph without cycles.
- **Dijkstra Relaxation:** if ($\text{dist}[w] > \text{dist}[u] + \text{cost}[u, w]$) then $\text{dist}[w] := \text{dist}[u] + \text{cost}[u, w]$.

3. Important Comparison Tables for 5/10-Mark Questions

A. Merge Sort vs Quick Sort

Parameter	Merge Sort	Quick Sort
Time Complexity (Worst)	$O(n \log n)$ (Guaranteed)	$O(n^2)$ (Occurs when sorted)
Auxiliary Space	$O(n)$ (Requires buffer array b)	$O(\log n)$ stack space (In-place)
Stability	Stable	Unstable
Work Phase	Major work done during Combine (Merge)	Major work done during Divide (Partition)

B. Prim's Algorithm vs Kruskal's Algorithm

Parameter	Prim's Algorithm	Kruskal's Algorithm
Approach	Grows a single continuous tree from a root	Merges disjoint trees in a forest
Best Suited For	Dense Graphs ($E \approx V^2$)	Sparse Graphs ($E \ll V^2$)
Cycle Prevention	Implicit (adds edges to non-tree nodes via <code>near[]</code>)	Explicit (Disjoint Sets Find-Set)
Time Complexity	$O(V^2)$ (Adjacency Matrix)	$O(E \log E)$ (Edge Sorting)

4. Algorithm Selection Guide (When to Use What)

Algorithm	When to Use (Scenario & Constraints)	When NOT to Use (Alternative)
Binary Search	When array is already sorted and fast $O(\log n)$ lookup is needed.	Unsorted data → Use Linear Search or sort first.
Merge Sort	When guaranteed $O(n \log n)$ and stability are required, or sorting linked lists / external files.	Strict memory limits → Use Quick Sort or Heap Sort.
Quick Sort	Fastest in practice for in-memory contiguous arrays; in-place $O(\log n)$ stack space.	Already sorted / adversarial inputs → Merge Sort.
Fractional Knapsack	Items can be divided into arbitrary fractions (fluids, grains, metal ingots).	Items are discrete (0/1) → Use Dynamic Programming.
Prim's Algorithm	Finding MST in dense graphs with many edges ($E \approx V^2$).	Sparse graphs with few edges → Use Kruskal.
Kruskal's Algorithm	Finding MST in sparse graphs ; when edges are already sorted.	Dense graphs → Use Prim.
Dijkstra's Algorithm	Single-source shortest paths in graphs with non-negative edge weights .	Graphs with negative edge weights → Bellman-Ford.
Job Sequencing	Unit-time jobs with deadlines and profits on a single processor.	Variable job execution times → Interval Scheduling.

5. Final University Exam Checklist

Ensure you have mastered every item before entering the examination hall:

◆ Exam Readiness Verification Checklist

UNIT-I: ALGORITHM BASICS & ANALYSIS

- Q1: Asymptotic Notations: Big-Oh (O), Omega (Ω), Theta (Θ), Little-oh (o) with definitions & proofs.
- Q2: Time Complexity formula $T(P)=C+T_p(I)$, program step definition & counting rules.
- Q3: Space Complexity formula $S(P)=C+S_p(I)$, fixed vs variable/stack space (abc, sum, RSum).
- Q4: Sequential Search Best $O(1)$, Worst $O(n)$, Average $O(n)$ on array [2, 5, 7, 8, 9].
- Q5: Largest of n numbers algorithm (max) and $\Theta(n)$ complexity.
- Q6: Tabular method tables: Iterative Sum ($2n+3$) and Matrix Addition ($2mn+2m+1$).
- Q7: Rate of growth: table for $F(n)=n^2+100n+\log_{10}n+1000$ & ordering $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
- Q8: 5 Characteristics of an algorithm: Input, Output, Definiteness, Finiteness, Effectiveness.
- Q9: 10 Pseudocode conventions (braces, $:=$, logical/relational, multidimensional arrays, loops, I/O).
- Q10: Recursive algorithms & factorial stack execution trace of $\text{rfactorial}(3)$.
- Q11: Matrix addition step count derivation ($2mn+2m+1$).
- Q12: Recursive sum recurrence $t(n)=2+t(n-1)$ derivation $\rightarrow 2n+2$ steps.
- Q13: Probabilistic analysis & Hiring Problem cost model ($n \cdot c_i + m \cdot c_h$) and $O(c_h \ln n)$.
- Q14: Connected components definition, DFS/BFS and $\{1,2,3,4\}$, $\{5,6\}$, $\{7\}$.
- Q15: Biconnected components, Articulation points (Cut Vertex 4) and BCC $1=\{0,1,2,3,4\}$, BCC $2=\{4,5,6,7,8\}$.

UNIT-II: DIVIDE AND CONQUER

- Q16: Divide and Conquer control abstraction DAndC(p) & general recurrence.
- Q17: Binary Search: algorithm, recurrence $T(n)=T(n/2)+c_2 \rightarrow O(\log n)$, Successful & Unsuccessful cases.
- Q18: Merge Sort: Divide, Conquer, Combine, trace on [6,2,8,4,3,7,5,1] and call tree.
- Q19: Merge Sort recurrence $T(n)=2T(n/2)+c_2n$ derivation $\rightarrow O(n \log n)$.
- Q20: Quick Sort: Partition procedure trace on [85,24,63,95,17,31,45,98] and algorithm.
- Q21: Quick Sort Best $O(n \log n)$ vs Worst $O(n^2)$ derivations.
- Q22: Strassen's Matrix Mult: motivation ($O(n^3)$ vs $O(n^2)$), 7 products $M_1 \dots M_7$, $C_{11} \dots C_{22}$ formulas.
- Q23: Strassen's recurrence $T(n)=7T(n/2)+c_2n^2$ derivation $\rightarrow O(n^{\log_2 7}) \approx O(n^{2.81})$.
- Q24: General Divide and Conquer recurrence $T(n)=aT(n/b)+f(n)$ & Master Theorem.
- Q25: Iterative Binary Search algorithm (BinSearch).
- Q26: Recursive Binary Search algorithm (BinSrch).
- Q27: Merge Sort 3 phases (Divide, Conquer, Combine).
- Q28: Complete Merge() algorithm with temporary array $b[]$.
- Q29: Complete QuickSort, Partition and interchange algorithms.
- Q30: Comparison table: Merge Sort vs Quick Sort.

UNIT-III: GREEDY METHOD

- Q31: Greedy Method control abstraction Greedy(a,n), greedy-choice property, feasible vs optimal.
- Q32: Fractional Knapsack: GreedyKnapsack algorithm & solved numerical ($w=[4,8,2,6,1]$, $p=[12,32,40,30,50]$, $m=10 \rightarrow$ profit \$124).
- Q33: Fractional vs 0/1 Knapsack comparison & counterexample ($m=50$, $w=[10,20,30]$, $p=[60,100,120] \rightarrow$ greedy \$160 vs optimal \$220).
- Q34: Spanning Tree definition & 4 key properties ($n-1$ edges, connects all, acyclic).
- Q35: Minimum Cost Spanning Tree (MST) definition & network wiring application.
- Q36: Prim's Algorithm: $\text{near}[]$ array role, pseudocode Prim(E, cost, n, t), 9-vertex graph solved trace (cost=28), $O(n^2)$.
- Q37: Kruskal's Algorithm: edge sorting, cycle check, pseudocode kruskal(E, cost, n, t), solved trace (cost=28), $O(E \log E)$.
- Q38: Disjoint Sets: Make-Set, Find-Set, Union operations and role in Kruskal.
- Q39: Single-Source Shortest Path (SSSP) problem definition & non-negative edge condition.
- Q40: Dijkstra's Algorithm: relaxation formula, pseudocode ShortestPaths, 6-vertex numerical trace table, $O(n^2)$.
- Q41: Job Sequencing with Deadlines: greedy slot selection, pseudocode JS(d, j, n), solved examples (4, 5, 7 jobs).
- Q42: Detailed comparison table: Prim's vs Kruskal's Algorithm.
- Q43: Solved MST numerical problem using Prim's algorithm (step-by-step table).
- Q44: Solved MST numerical problem using Kruskal's algorithm (step-by-step table).
- Q45: Solved Dijkstra shortest-path numerical problem (iteration table).
- Q46: Solved Job Sequencing with Deadlines numerical problem (slot assignment table).